SOLID Principles in Software Engineering

Dr. Adnan Arefeen

Dept. of ECE
North South University

Any problem with this code?

```python
class Invoice:
    def __init__(self, items):
        self.items = items

    def calculate_total(self):
        return sum(item['price'] for item in self.items)

    def print_invoice(self):
        print(f"Total: {self.calculate_total()}")
```

Any problem with this code?

```python
class Invoice:
    def __init__(self, items):
        self.items = items

    def calculate_total(self):
        return sum(item['price'] for item in self.items)

    def print_invoice(self):
        print(f"Total: {self.calculate_total()}")
```

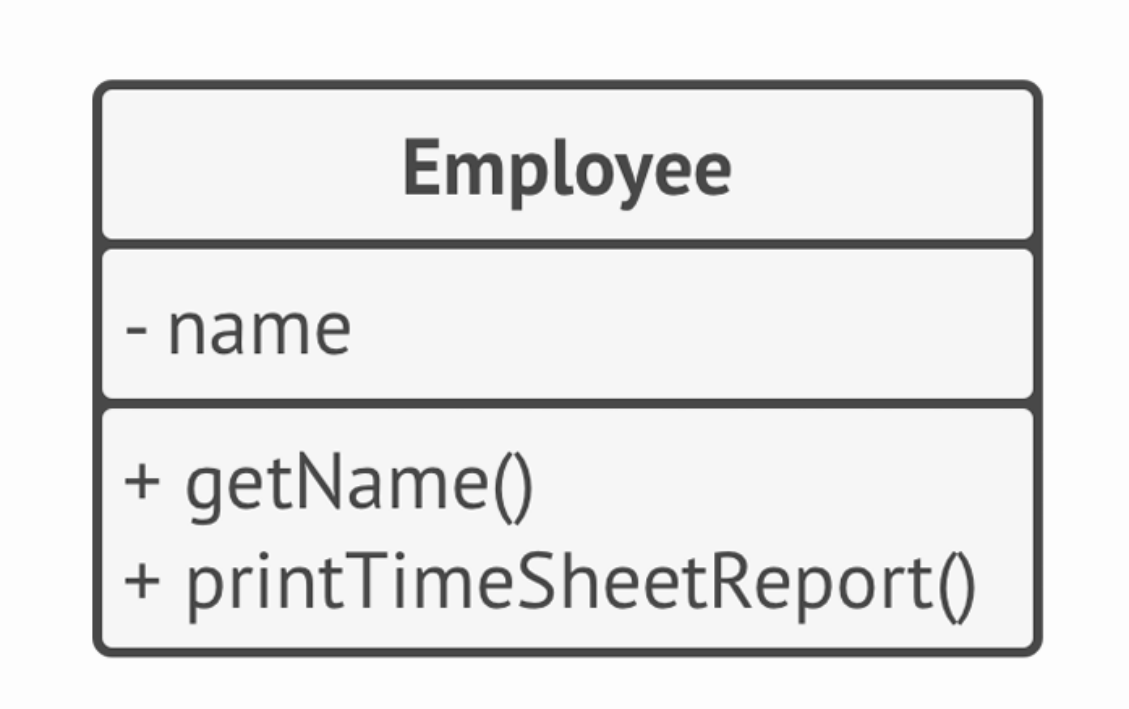
If you need to change how printing works (e.g., export to PDF or GUI), you'll have to modify this class: risking changes to unrelated functionality (calculation).

Single Responsibility Principle (SRP)

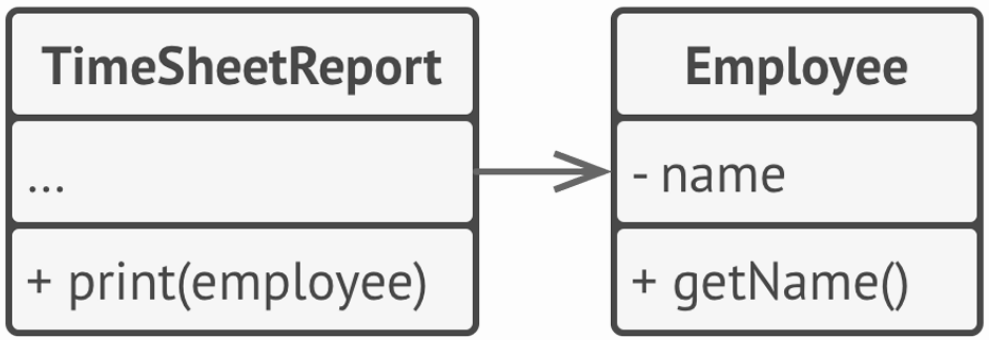
A class should have only one reason to change.

```
class Invoice:
    def __init__(self, items):
        self.items = items
    def calculate_total(self):
        return sum(item['price'] for item in self.items)

class InvoicePrinter:
    def print_invoice(self, invoice):
        print("Total:", invoice.calculate_total())
```

Does it follow SRP?

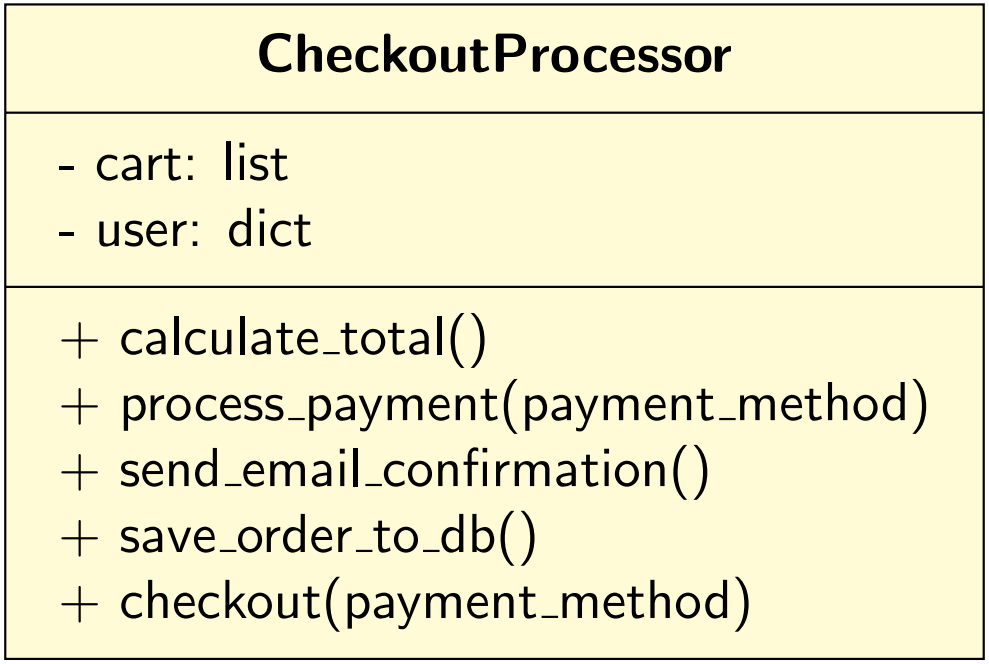


Does it follow SRP? Solution

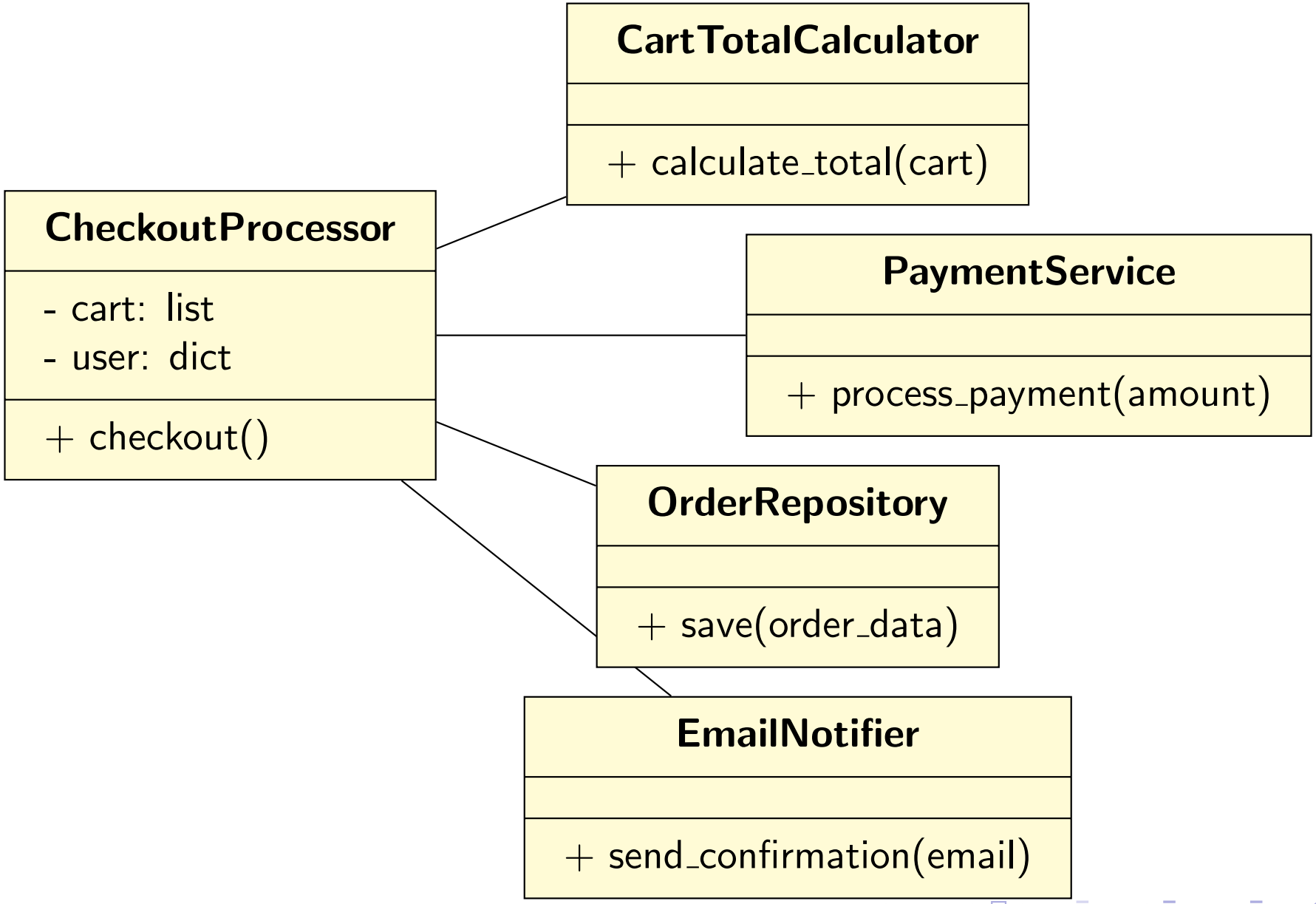


AFTER: the extra behavior is in its own class.

Good SRP example?



Good SRP example?



Any problem with this code?

```python
class Notifier:
    def send(self, message, method):
        if method == 'email':
            print(f"Sending Email: {message}")
        elif method == 'sms':
            print(f"Sending SMS: {message}")
        elif method == 'slack':
            print(f"Sending Slack message: {message}")
        else:
            raise ValueError("Unknown method")
```

Any problem with this code?

```python
class Notifier:
    def send(self, message, method):
        if method == 'email':
            print(f"Sending Email: {message}")
        elif method == 'sms':
            print(f"Sending SMS: {message}")
        elif method == 'slack':
            print(f"Sending Slack message: {message}")
        else:
            raise ValueError("Unknown method")
```

Problem:

- Adding new channels requires modifying this class.
- Risk of bugs in existing logic.

Open/Closed Principle (OCP)

- Software entities should be open for extension
- Closed for modification.

Step 1 — Define Abstract Interface

```python
from abc import ABC, abstractmethod

class NotificationSender(ABC):
    @abstractmethod
    def send(self, message):
        pass
```

Step 1 — Define Abstract Interface

```python
from abc import ABC, abstractmethod

class NotificationSender(ABC):
    @abstractmethod
    def send(self, message):
        pass
```

Each channel will implement this interface.

Step 2 — Add Notification Senders

```python
class EmailSender(NotificationSender):
    def send(self, message):
        print(f"Sending Email: {message}")

class SMSSender(NotificationSender):
    def send(self, message):
        print(f"Sending SMS: {message}")

class SlackSender(NotificationSender):
    def send(self, message):
        print(f"Sending Slack: {message}")
```

Step 3 — Refactor Notifier

```python
class Notifier:
    def __init__(self, sender: NotificationSender):
        self.sender = sender

    def notify(self, message):
        self.sender.send(message)
```

Step 3 — Refactor Notifier

```python
class Notifier:
    def __init__(self, sender: NotificationSender):
        self.sender = sender

    def notify(self, message):
        self.sender.send(message)
```

Now we can extend without modifying Notifier.

Step 4 — Final Usage

```
notifier = Notifier(EmailSender())
notifier.notify("Project deadline approaching!")

notifier = Notifier(SlackSender())
notifier.notify("Server is down!")
```

Step 4 — Final Usage

```
notifier = Notifier(EmailSender())
notifier.notify("Project deadline approaching!")

notifier = Notifier(SlackSender())
notifier.notify("Server is down!")
```

OCP achieved! Notifier stays untouched as new channels are added.

Liskov Substitution Principle (LSP)

When extending a class, remember that you should be able to pass objects of the subclass in place of objects of the parent class without breaking the client code.

LSP and Method Parameters

Rule: In a subclass, method parameter types should match or be more general than in the superclass.

Original class:

- `feed(Cat c)` – expects any `Cat`.

Good subclass:

- `feed(Animal c)` – accepts `Animal`, so still accepts `Cat`.

Bad subclass:

- `feed(BengalCat c)` – only accepts a specific subtype; breaks code that passes generic `Cat`.

Result: The client code fails if the subclass narrows the parameter type.

LSP and Method Return Types

Rule: In a subclass, the return type should match or be a subtype of the return type in the superclass.

Original class:

- `buyCat()`: `Cat` – client expects a `Cat`.

Good subclass:

- `buyCat()`: `BengalCat` – returns a subtype of `Cat`, everything works.

Bad subclass:

- `buyCat()`: `Animal` – returns a general type; could break code expecting `Cat`-specific behavior.

Also problematic:

- Base returns a `String`, override returns a `Number` – incompatible in dynamically typed languages.

Result: The client code breaks if the return type becomes more general or unrelated.

Problems in Design?

```
class Bird:
    def fly(self): pass

class Duck(Bird):
    def fly(self): print("Duck flying")

class Ostrich(Bird):
    def fly(self): raise Exception("Can't fly")
```

Problems in Design?

```
class Bird:
    def fly(self): pass

class Duck(Bird):
    def fly(self): print("Duck flying")

class Ostrich(Bird):
    def fly(self): raise Exception("Can't fly")
```

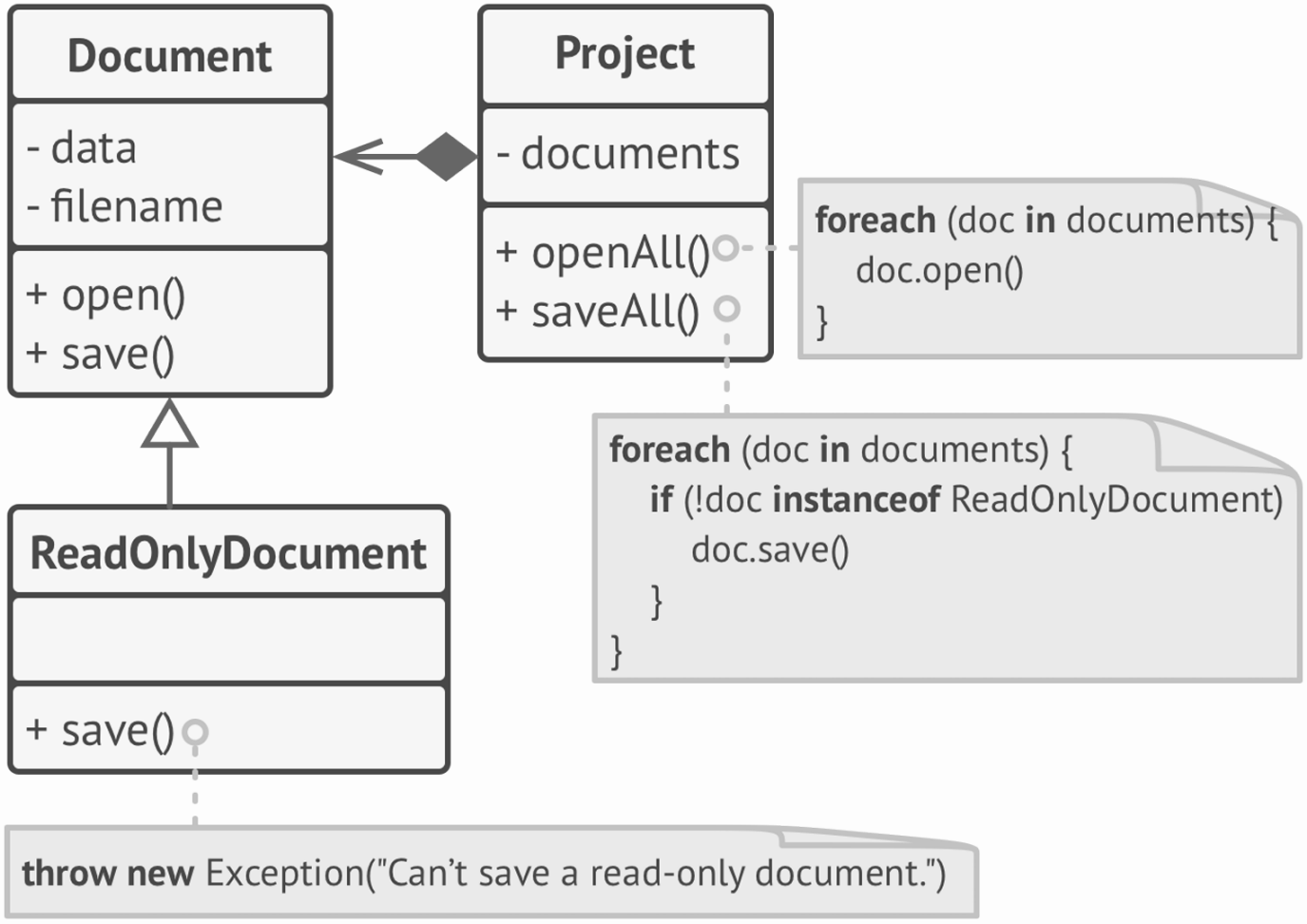
A method in a subclass should not throw an exception.

Better LSP Design

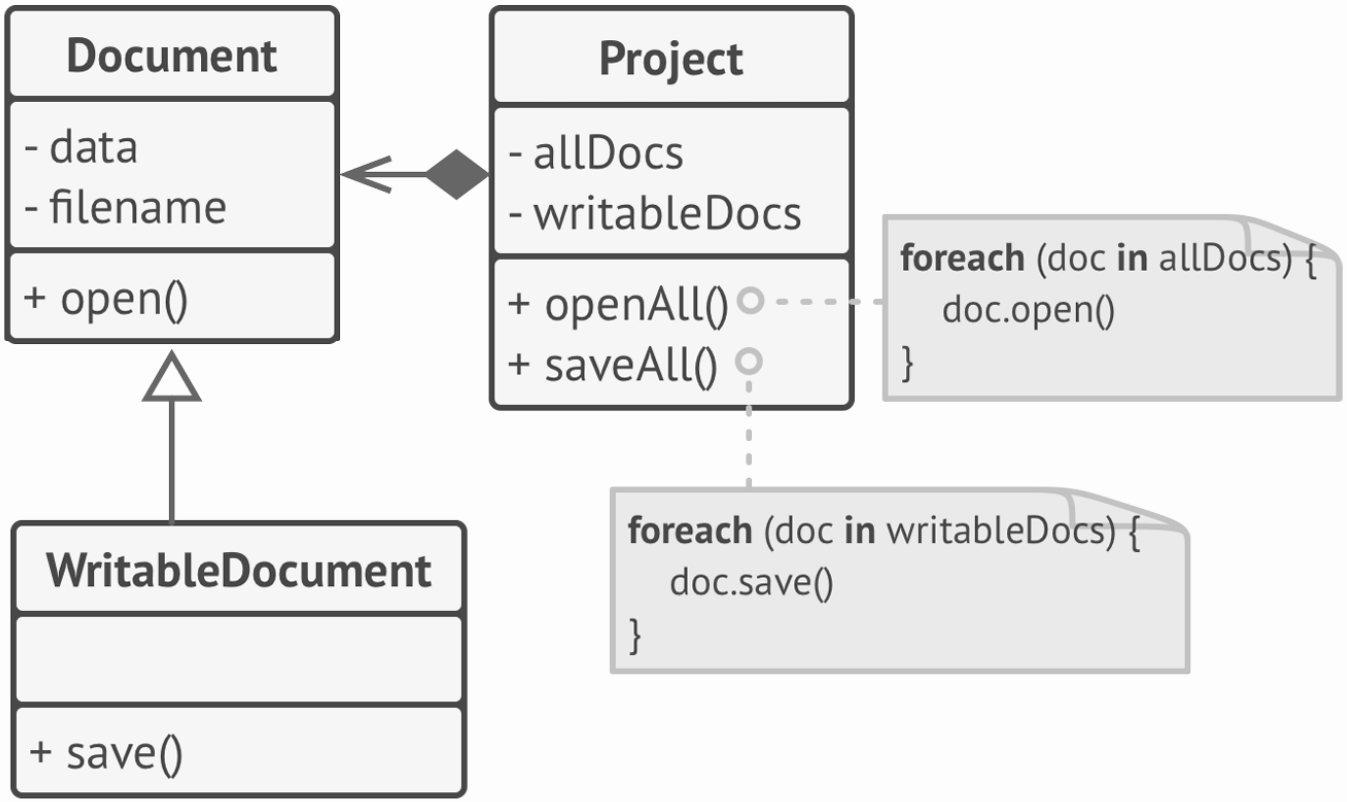
Separate flight capability into another class.

```
class Bird: pass

class FlyingBird(Bird):
    def fly(self): print("Flying")

class Duck(FlyingBird): pass
class Ostrich(Bird):
    def walk(self): print("Walking")
```

Any problem with this setup?

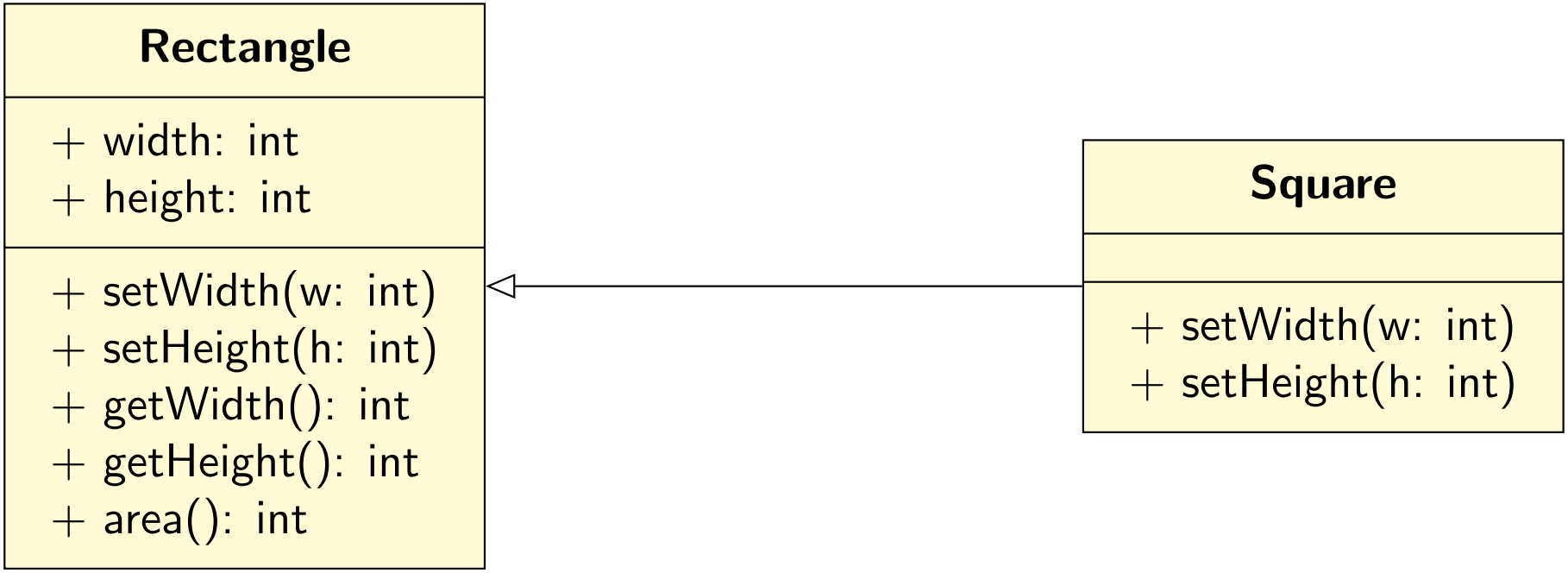


Solution



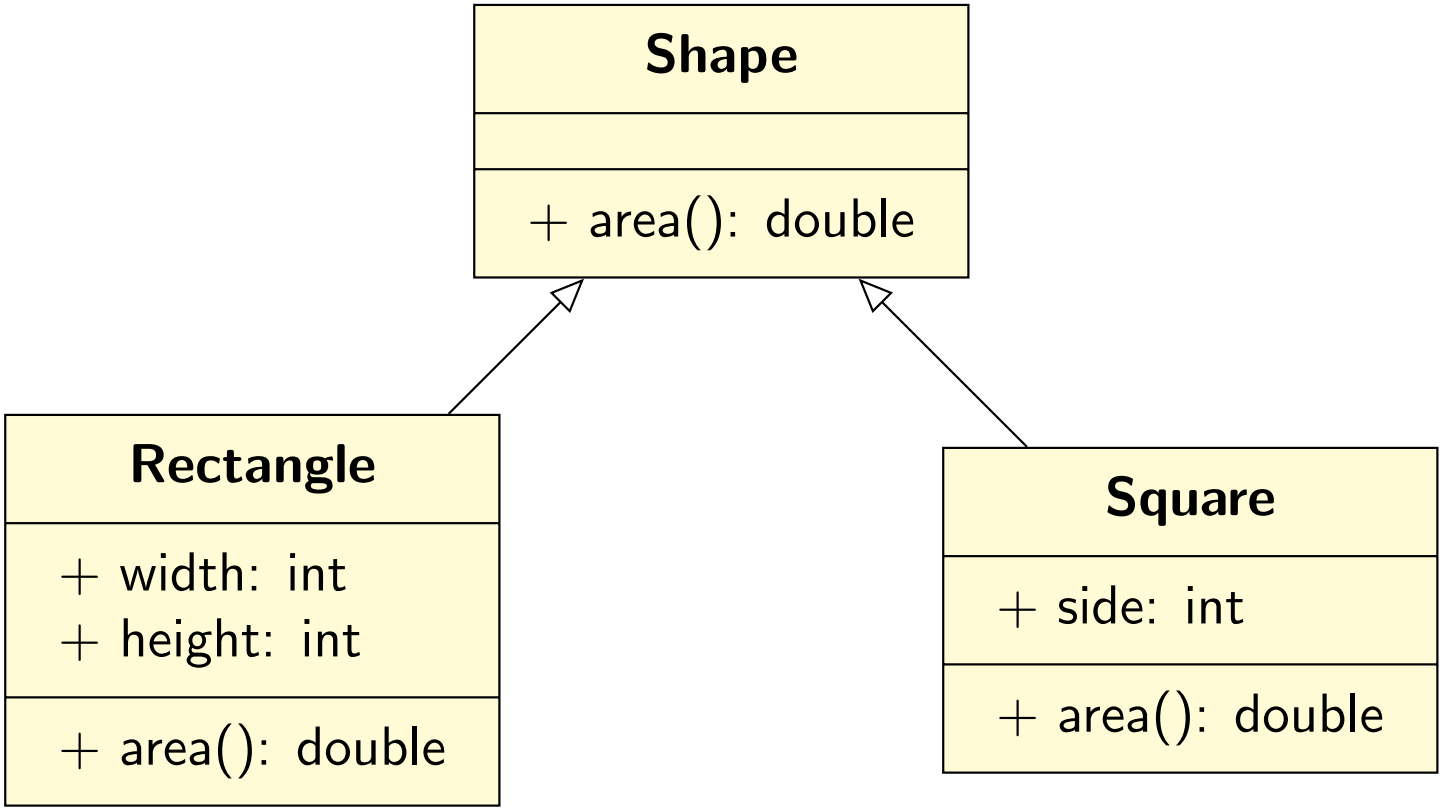
AFTER: the problem is solved after making the read-only document class the base class of the hierarchy.

Problematic Design: Rectangle → Square (LSP Violation)



Issue: `Square` overrides methods in a way that breaks the expected behavior of `Rectangle`, violating LSP.

LSP-Compliant Design: Common Shape Interface



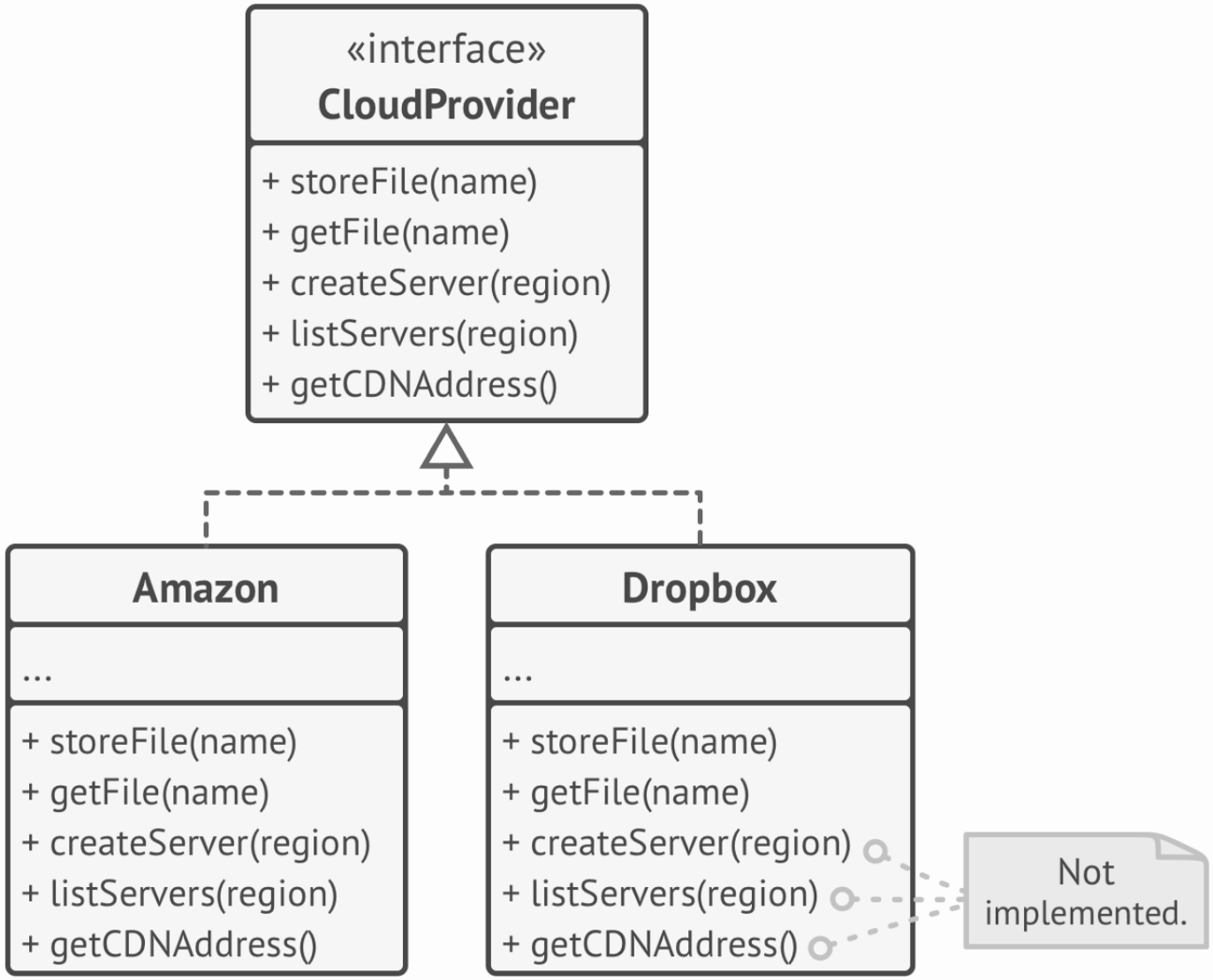
Solution: Use a base `Shape` interface and implement `area()` individually in `Rectangle` and `Square`. Substitution is now safe.

Interface Segregation Principle (ISP)

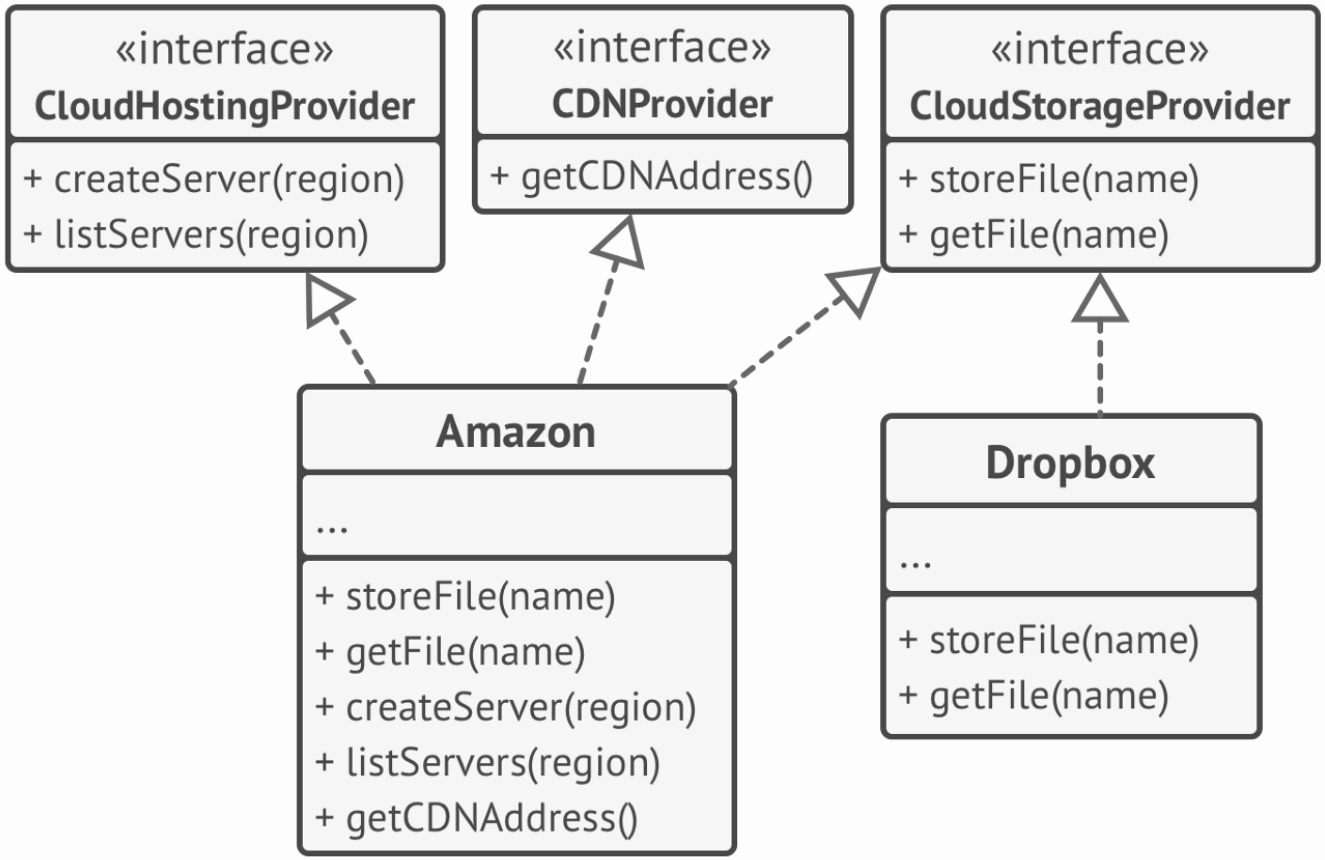
Clients should not depend on interfaces they do not use.

```
class IPrinter:
    def print(self): pass

class IScanner:
    def scan(self): pass

class AllInOnePrinter(IPrinter, IScanner):
    def print(self): print("Printing")
    def scan(self): print("Scanning")
```

Any problem with this setup?



Solution



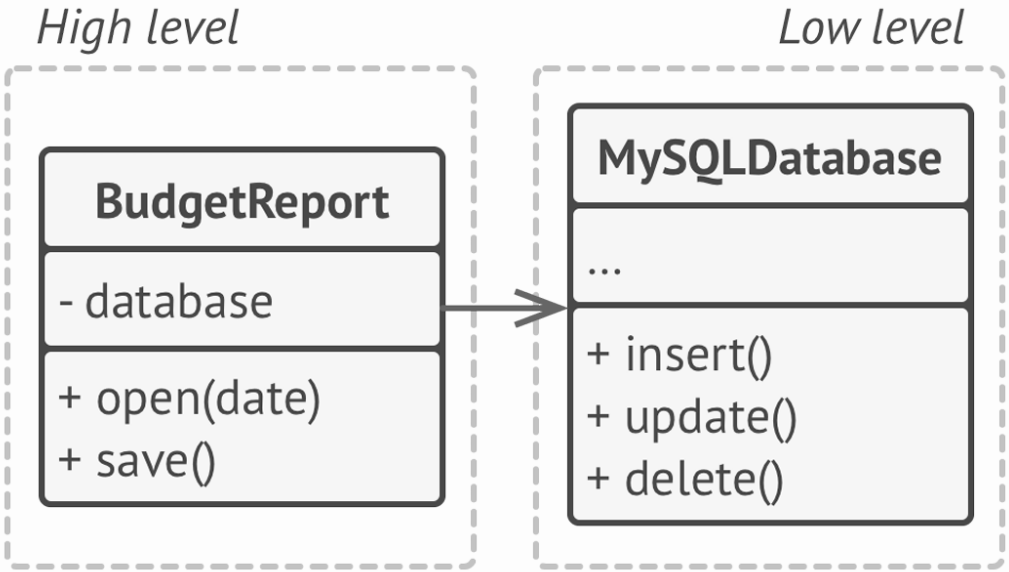
AFTER: one bloated interface is broken down into a set of more granular interfaces.

Dependency Inversion Principle (DIP)

Depend on abstractions, not on concretions.

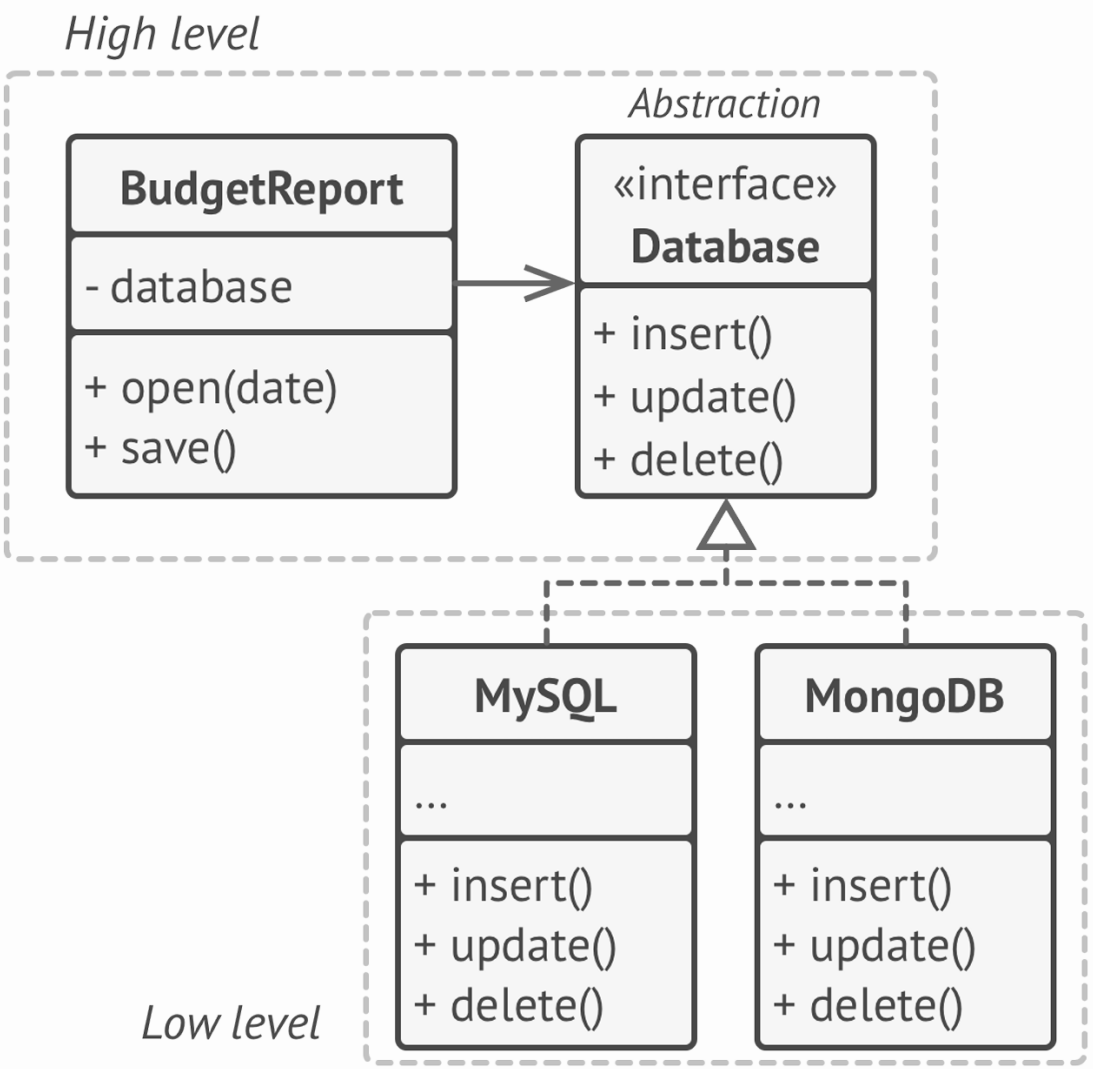
```python
class IMessageService:
    def send(self, message): pass

class EmailService(IMessageService):
    def send(self, message): print(f"Email sent: {message}")

class Notification:
    def __init__(self, service):
        self.service = service
    def alert(self, msg):
        self.service.send(msg)
```

Any problem with this setup?



BEFORE: a high-level class depends on a low-level class.

Solution



AFTER: low-level classes depend on a high-level abstraction.